

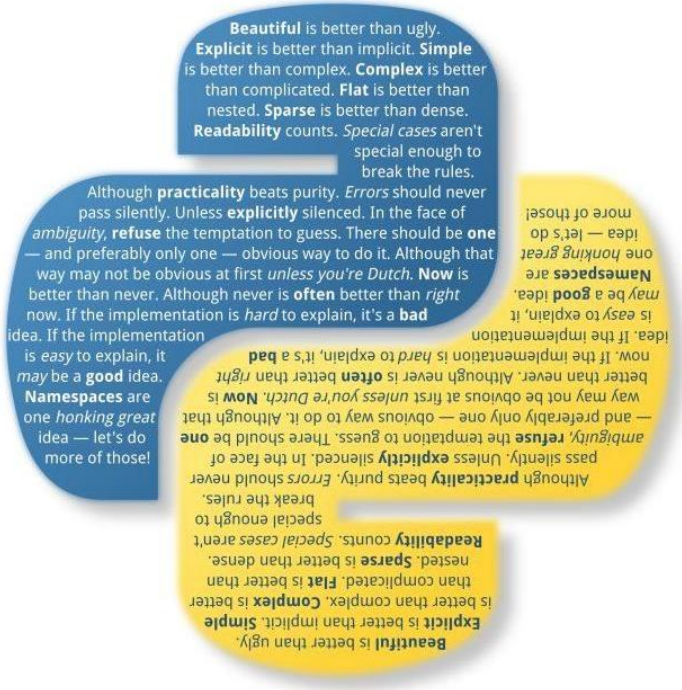
计算机辅助制造实践

——Python实现三维打印路径规划

浙江大学机械工程学院 林志伟
玉泉教1-215　zjtzhylin1986@163.com

第1章 Python简介

- Python语言概述
- Python解释器安装及运行
- Python第三方库安装
- Python集成开发环境
- Python语法简介
- Python模块



python™

1. Python语言概述

- Python的创始人为**Guido van Rossum**。
  - 1989年圣诞节期间，在荷兰的阿姆斯特丹，Guido为了打发圣诞的无聊时间，决心开发一个新的解释型脚本语言——Python。
  - 他希望Python能够实现他的理念，成为一种介于C语言和Shell之间，功能全面，易学易用，拓展性强的语言。

——英国喜剧：《蒙提·派森的飞行马戏团》（Monty Python and the Flying Circus）



Python创始人
Guido van Rossum

- 1991年，第一个Python**解释器**诞生。它用C语言实现，并能调用C库。从一出生，Python就已具备了类（class）、函数（function）、异常处理（exception），包含表（list）和词典（dictionary）在内的核心数据类型，以及模块（module）为基础的拓展系统。



Guido弟弟设计的早期Python logo

1. Python语言概述

- **经过近30年的发展，Python逐渐完善，优点包括：**
  - ✓ **简单易学**。Python崇尚优美、清晰、简洁，是一种代表简单主义思想的语言，拥有极其简单的语法，即便是没有任何编程基础的人，也非常容易上手；且Python适合阅读，就像是在读英语一样。
  - ✓ **免费开源**。Python是FLOSS（自由/开放源码软件）之一，用户可以自由地发布这个软件的拷贝、阅读它的源代码、对它做改动、把它的一部分用于新的自由软件中。
  - ✓ **丰富的库**。Python 类库非常全面，对大都功能一般都有现成的类库可以使用；如果一个功能比较特殊，标准库没有提供相应的支持，那么，很大概率也会有相应的开源项目提供了类似的功能。
  - ✓ **可移植性**。同一份Python代码，无需大的修改，便可运行在众多平台上，包括Linux、Windows、FreeBSD、Macintosh、Solaris、OS/2、Amiga、Windows CE甚至还有PocketPC、Symbian以及Google的Android平台。
- **但是，Python仍然存在一些缺点，包括：**
  - **运行速度慢**。比C/C++的性能差一个数量级。
  - **高低版本不兼容**。Python 2 和Python3 是两个平行发展的版本，让用户莫衷一是！
  - **多处理器支持差**。

1. Python语言概述

- Python和其他编程语言**性能对比**

案例：在同一平台上，求0~200000之整数区间的质数个数。	
C	**6s**
Java	7s
Nodejs	8s
Golang	17s
Python	**121s**

针对这个案例
C语言的执行效率约是Python的20倍

- Python运行效率不高，但是其**开发效率很高。**

——这就是Python的优势。

1. Python语言概述

- Python应用领域
  - **云计算开发** **OpenStack**
  - **Web开发** **Django**
  - **科学运算** **NumPy、SciPy、Matplotlib**
  - **人工智能** **PyTorch、Tensorflow**
  - ……

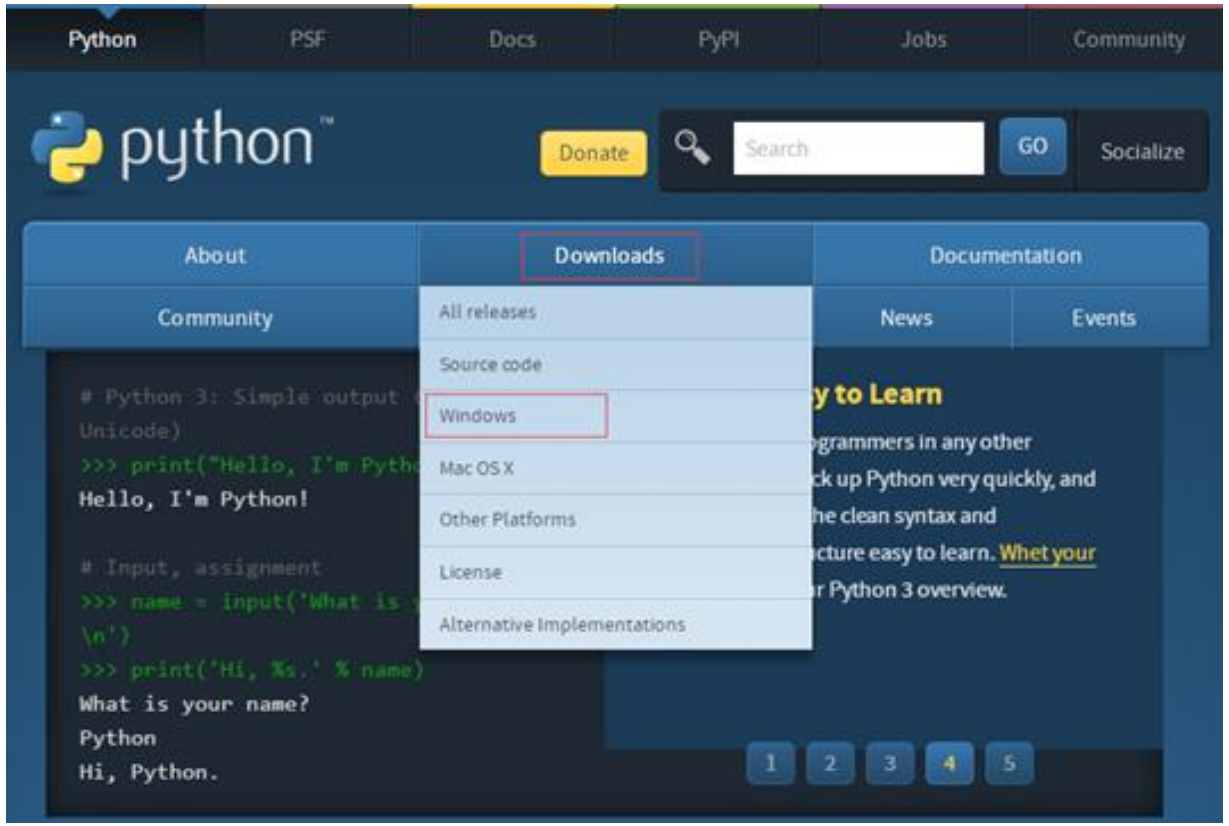


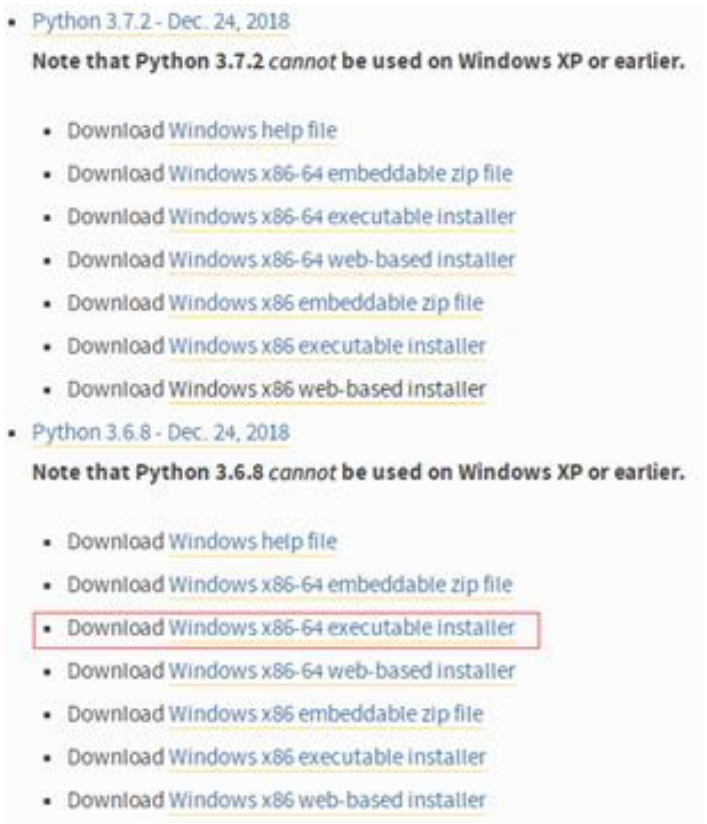
- 特别是**人工智能**领域，当AI时代来临后，Python从众多编程语言中脱颖而出，大都人工智能算法是基于Python编写或提供Python接口。



2. Python解释器安装及运行

- **下载Python解释器**
  - 登录Python官网（https://www.python.org/），点击Downloads菜单下的**Windows**菜单项，进入Windows版Python编译器下载页面（图左）。在弹出的下载页面中（图右），根据实际需要下载相应的Python版本。





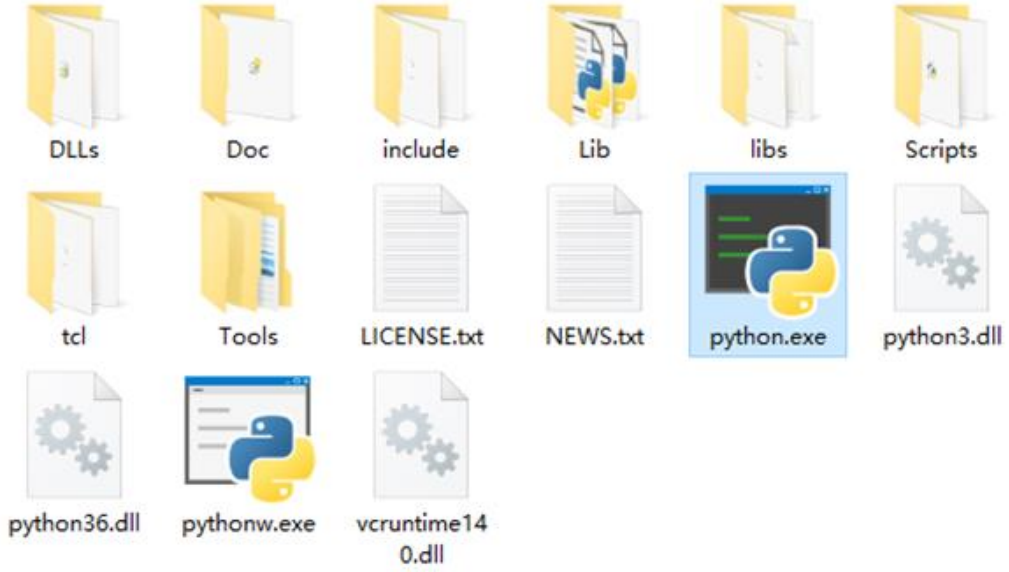
2. Python解释器安装及运行

• 安装Python解释器

- 双击安装包，弹出Python for Windows安装界面。在该界面上，有两种安装方式可选，默认安装（Install Now）和自定义安装（Customize installation）。
- 对初学者来说，首选默认安装，点击Install Now后，安装程序会把相关组件安装到系统盘默认账户文件夹下。
- 如果选择自定义安装，点击Customize installation后，进入自定义安装界面，在这些界面上可配置需要安装的组件及安装目录，如果无特殊需求，一般只需更改程序安装目录即可，如可将程序安装至 D:\Program Files\Python36\ 中。



安装界面（注意勾选环境变量）



安装目录下的文件

2. Python解释器安装及运行

• 运行Python解释器

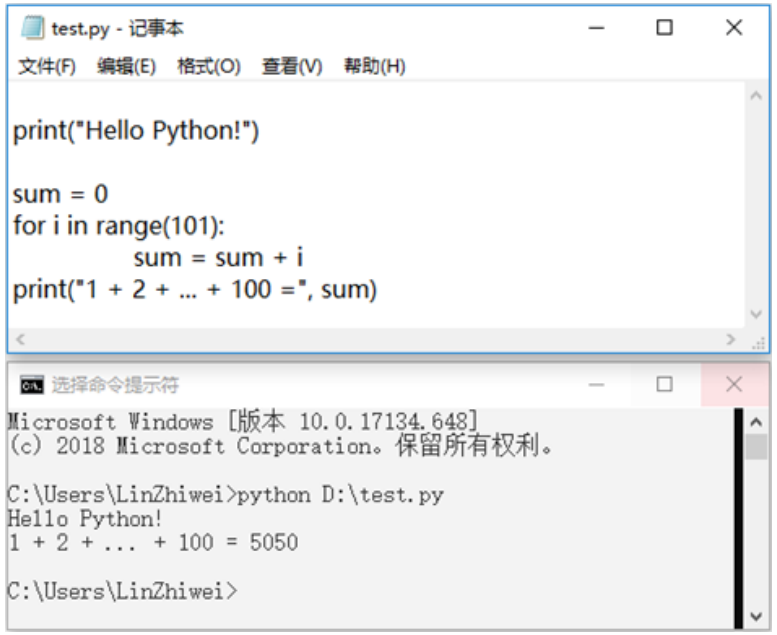
- 双击安装目录下的python.exe启动Python解释器，此时窗口提示已经安装的Python版本信息。
- 输入“print(“Hello Python!”)”指令，命令窗口打印“Hello Python！”字符，这表明我们已经开启了Python编程之旅!
- 如果环境变量设置正确，只需在Windows命令窗口输入“python”，同样可以启动Python解释器。输入“python -V”可以获得当前安装的Python版本号。



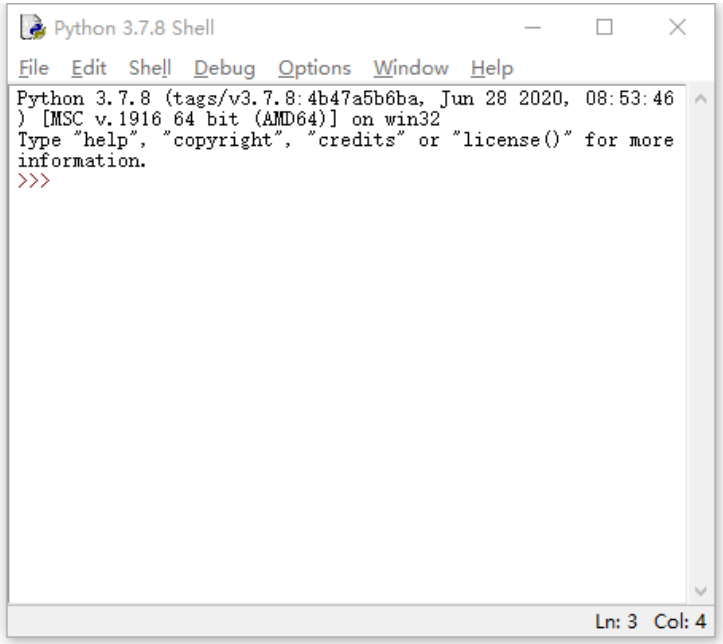
2. Python解释器安装及运行

• 运行Python脚本

- 对于结构复杂的程序，一般会将所需实现的业务逻辑编写到**脚本**程序中，然后通过Python解释器运行脚本程序。
- 如可在D盘根目录下创建一个test.py文本文件，在里面编写相应脚本程序，然后在Windows命令窗口通过“python D:\test.py”运行该脚本。
- 此外，也可通过Python自带的集成开发环境**IDLE**运行脚本程序。



运行Python脚本



Python自带IDE

3. Python第三方库安装

- Python第三方库安装非常简单，只需要在命令窗口输入“**pip install 库名**”即可。比如需要安装串口库，只需在命令窗口输入“pip install pyserial”。
- pip install 命令会自动识别当前系统上正在使用的Python版本，并根据版本信息自动去相关网站搜索目标第三方库，然后将合适（兼容）的第三方库下载、安装到本地。安装完成后，命令窗口将提示“Successfully installed 库名”。
- 如果需要卸载已经安装的第三方库，只需在命令窗口输入“**pip uninstall 库名**”，比如需要卸载pyserial库，只需输入“pip uninstall pyserial”即可。
- 如果需要在代码文件使用该库，只需在文件开头添加“**import 库名**”即可，如需使用串口库，只需在文件开头添加一行“import serial”代码即可。



4. Python集成开发环境

- Python的集成开发环境（IDE）很多，如：Vim、Eclipse with PyDev、Sumlime Text、PyCharm、Wing、Visual Studio Code、Anaconda等等。
  - 这些IDE大同小异，功能强大，一般都包含代码调试、语法高亮、项目管理、智能提示、代码补全、代码跳转等功能。
  - PyCharm是由JetBrains打造的一款Python IDE，推荐使用**PyCharm**作为Python开发工具。
- PyCharm的官方下载地址为http://www.jetbrains.com/pycharm/。PyCharm目前提供**专业版（Professional）**和**社区版（Community）**两个不同版本。其中专业版包含了PyCharm IDE的所有功能，适合Python及Web专业开发人员；社区版为轻量级IDE，适合科研人员或初学者使用。
- 专业版收费，但提供30天免费试用；社区版完全免费且开源。考虑到社区版功能已经够用，我们可选择使用社区版PyCharm，下载最新发布的PyCharm Community即可。
- PyCharm安装较为简单，如果对其安装目录没有要求，一路点击下一步默认安装即可。安装完成后，在桌面点击pycharm快捷方式启动PyCharm软件。首次启动时，需要对IDE进行配置（如配色），选择默认配置即可。

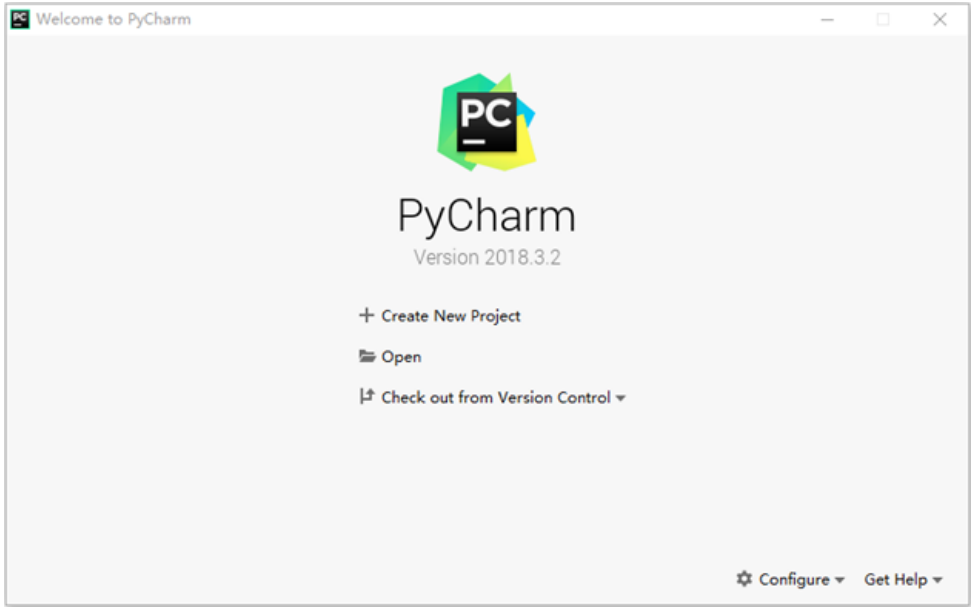


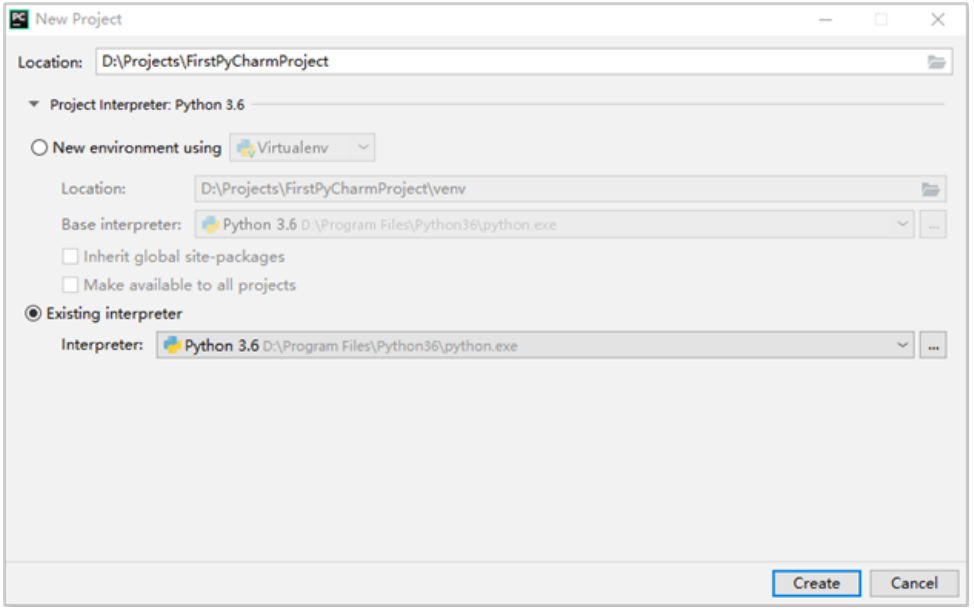
http://www.jetbrains.com/pycharm/

4. Python集成开发环境

• 新建项目

- PyCharm启动后，点击**Create New Project**，在新页面中，首先在Location中输入新项目存放目录及项目名。这里将项目放在D:\Projects\目录下，项目名为FirstPyCharmProject，然后在Project Interpreter中选择Python解释器。
- 这一步建议选择Existing Interpreter选项，并在Interpreter中选择之前安装的Python解释器文件路径。如果选择New environment选项，PyCharm会将Python解释器及相应库文件拷贝至项目目录，这样只会增加项目文件尺寸。
- 点击Create按钮，创建项目并进入编程PyCharm主界面。

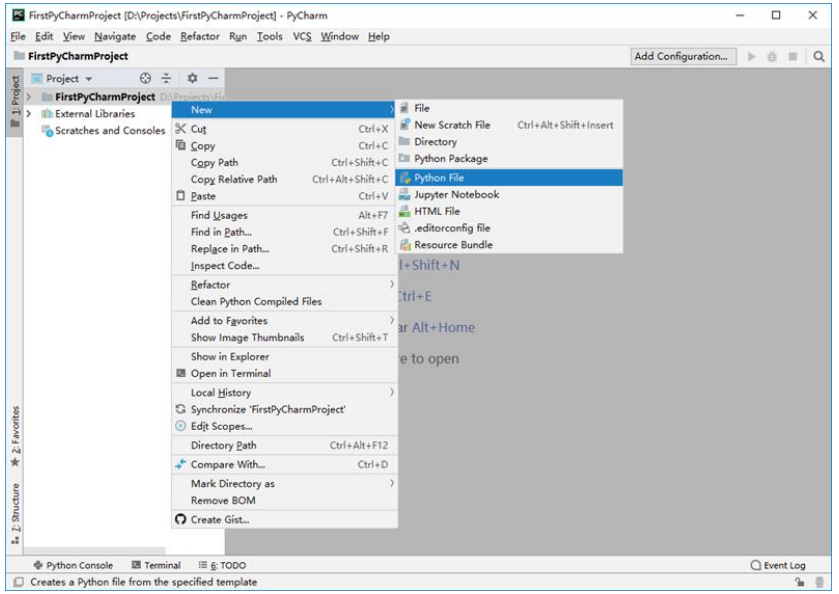


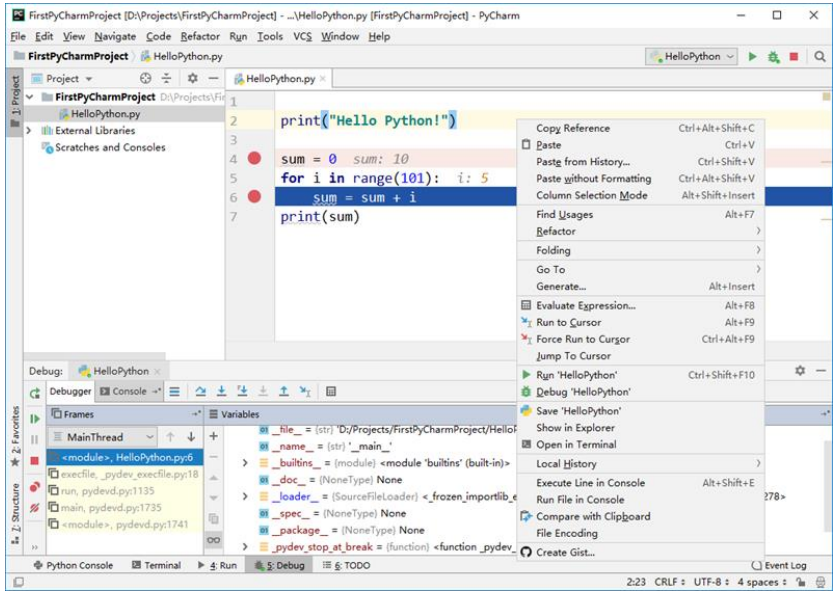


4. Python集成开发环境

• 运行项目

- 鼠标右键点击管理窗口中的项目文件夹，在弹出菜单中选择New→Python File，然后在弹出的对话框中输入脚本文件名，这里以HelloPython为例输入。
- 在代码窗口，可以输入目标代码，这里以打印“Hello Python！”和求1到100累加和代码为例。在左侧项目管理窗口，我们可以按实际需要添加一个或多个能够实现不同功能的Python脚本文件。
- 首次运行代码需选中左侧项目管理窗口中的相应脚本文件或右侧代码窗口中已经打开的脚本文件，点击鼠标右键，在弹出菜单中选择**Run** ‘HelloPython’或**Debug** ‘HelloPython’，即可运行HelloPython脚本程序。





5. Python语法简介

- **数据类型**
  - Python内置了5种标准数据类型，分别为：
    - **Numbers** **数字**
    - **String** **字符串**
    - **List** **列表**
    - **Tuple** **元组**
    - **Dictionary** **字典**
  - 其中，Numbers又分为4种类型，分别为：
    - **Int** **整型**
    - **Bool** **布尔型**
    - **Float** **浮点型**
    - **Complex** **复数**

5. Python语法简介

- **数据类型**

- Python是**弱类型语言**，**不需要声明类型**，其类型由其赋值的类型决定；变量名首字符必须为字母或下划线_（不能为数字），建议使用小写英文字母开头。

	变量赋值操作	1-1
1	counter = 100	# 整型变量
2	miles = 1000.0	# 浮点型变量
3	city= “Hangzhou”	# 字符串变量，双引号和单引号可以通用
4	flag = True	# 布尔型变量，值为 True 或 False
5	a = b = c = 1	# 为多个变量赋同一值
6	name, age, sex = ‘张三’, 23, ‘男’	# 为多个变量赋不同（类型）值
7	names = [‘Jim’, ‘Tom’, ‘Kate’]	# 定义列表变量，并为其赋值
8	classes = (‘English’, ‘math’, ‘physics’)	
9	students = {‘Jim’ : 10, ‘Tom’ : 11, ‘Kate’ : 9}	# 定义字典变量，并为其赋值

5. Python语法简介

- **数据类型：列表、元组、字典**
  - 列表定义使用方括号[]，元组定义使用圆括号()，字典定义使用花括号{}。
  - 列表和元组每个元素就是一个值，字典每个元素为一个**键值对（key: value）**。列表和元组的不同点在于列表中的元素个数（及元素类型）可以在定义后动态改变，而元组中的元素个数（及元素类型）在定义后不能更改。字典中的元素个数（及元素类型）也可动态改变。

列表和字典操作 1-2

```
1  # 列表
2  names = []                          # 定义一个空列表变量
3  names.append('Jim')                 # 为列表添加元素
4  names.append('Tom')                 # 为列表添加元素
5  names.append('Kate')                # 为列表添加元素
6  names.remove('Tom')                 # 按值删除元素，删除列表中首个满足给定值的元素
7  del names[1]                        # 按索引号删除元素
8  names.clear()                       # 清空列表
9  # 字典
10 students = {}                       # 定义一个空字典变量
11 students['Jim'] = 10                # 为字典添加键值对，其中Jim为主键，10为值
12 students['Tom'] = 11                # 如果不包含Tom键，则添加；如果已包含，则修改值
13 students['Kate'] = 9
14 del students['Tom']                 # 根据主键删除字典中元素
15 students.clear()                    # 清空字典
```

5. Python语法简介

- **数据类型：列表、元组、字典**
- 列表（或元组）、字典中各个元素的遍历（读取）方法如下。

	列表（或元组）、字典遍历方法	1-3

```
1  # 列表遍历方法
2  names = ['Jim', 'Tom', 'Kate']        # 定义names列表变量，并为其赋值
3  for x in names:
4      print(x)                          # Python中采用缩进表示代码块，缩进使用tab键实现
5  # 字典遍历方法
6  students = {'Jim' : 10, 'Tom' : 11, 'Kate' : 9}      # 定义students字典变量
7  # 遍历字典主键
8  for key in students.keys():
9    print(key, students[key])           # 打印主键和主键对应的值
10 # 遍历字典值
11 for val in students.values():
12   print(val)                          # 打印值
13 # 直接遍历键值对
14 for key, val in students.items():
15   print(key, val)                     # 打印键值对
```

5. Python语法简介

• 运算符

- Python内建加、减、乘、除等数值运算符，大于、小于、等于等关系运算符，以及与、或、非三个逻辑运算符。

类别	运算符	描述
数值	+	加：两个对象相加
	-	减：得到负数或是一个数减去另一个数
	*	乘：两个数相乘或是返回一个被重复若干次的字符串
	/	除：两个数相除
	%	取模：返回除法的余数
	//	取整除：返回商的整数部分
	**	幂：返回x的y次幂
关系	==	等于：比较对象是否相等
	!=	不等于：比较两个对象是否不相等
	>	大于：返回x是否大于y
	<	小于：返回x是否小于y。
	>=	大于等于：返回x是否大于等于y。
	<=	小于等于：返回x是否小于等于y。
逻辑	and	与：x and y，两个同时为True时为True
	or	或：x or y，两个只要有一个True即为True
	not	非：not y，取反

5. Python语法简介

- **流程控制**：顺序、**选择**、循环

- Python中的选择结构使用if、elif、else三个关键字引导。如果程序只有两个分支，使用if、else即可；如果程序包含两个以上分支，需要使用elif。(注意：只要有一个条件成立，就会执行其后的语句，执行完后程序将退出整个if语句，而不会继续执行其他elif或else语句！)

选择结构，if、elif、else语句 1-4

```
1  score = int(input("请输入您的总分："))          # 输入分数
2  if score >= 95:                                 # if 条件后输入后必须输入冒号
3    print('优')
4  elif score >= 85:
5    print('良')
6  elif score >= 70:
7    print('中')
8  elif score >= 60:
9    print('合格')
10 else:
11   print('不及格')
```

- Python语言的**三目运算符**，使用一行代码就可实现二选一功能:

```
1  x if b else y                    # Python中的三目运算
```

```
1  b ? x : y                        # C++、C#、Java等语言中的三目运算
```

5. Python语法简介

- **流程控制**：顺序、选择、**循环**

- 在Python中，循环结构使用for或while关键字引导，其中for循环一般用来引导循环次数已知的程序代码，while循环一般用来引导循环结束条件已知的程序代码。

循环结构，for语句和while语句 1-5

```
1  # 寻找100以内质数并打印
2  for number in range(1, 101):                    # for经常和range函数一起使用，range生成遍历范围
3      isPrime = True
4      for i in range(2, number):
5          if number % i == 0:                     # 如果余数为0，则i为number的因数，number为合数
6              j = number / i                          # 算出number的另一个因数
7              print("%d = %d * %d" % (number, i, j))  # 打印合数
8              isPrime= False
9              break                                   # 退出当前层for循环
10     if isPrime:
11         print("%d 是一个质数" % num)          #如果number没有一个因数，则为质数
12 # 循环结构，while语句，寻找100以内5的倍数并打印
13 val = 0                                          # 初始化val值为0
14 while (val <= 100):                             # 循环结束条件为val <= 100
15     val = val + 1
16     if val % 5 == 0:
17         print(val)
18     else:
19         continue                                # 如果不满足条件则继续循环
```

注：range(1，101)函数表示生成1到100，步长为1的列表；如果需要生成步长为2的列表，只需输入range函数的第三个参数即可：range(1，101，2)。

5. Python语法简介

- **函数**

- 对一般编程语言，创建一个函数需要定义函数**返回类型**、**函数名**、**参数**（包括参数个数、参数类型、参数名）以及**函数体**。因为Python是弱类型语言，在定义函数过程中，**不需要定义函数返回类型以及参数类型**。

	函数定义和调用	1-6
1	# 定义两个数求和函数	
2	**def add(a, b):**	# def不可或缺，为函数定义关键字；add为函数名，a，b为参数
3	sum = a + b	# 函数体，对参数a和b求和，sum保存结果
4	return sum	# 返回运算结果
5	# 函数调用，两数求和	
6	x = 1	
7	y = 2	
8	z = add(x, y)	
9	print(z)	# 输出3
10	# 该函数同样适用于字符串拼接	
11	u = 'I am '	
12	v = 'a student. '	
13	w = add(u, v)	
14	print(w)	# 输出“I am a student.”

- 由于以上编写的add函数**参数a、b类型不定**，可以输入整数，也可以输入浮点数，甚至可以输入字符串，而返回的结果总是我们想要的，这正是Python的魅力之一！事实上，这有点像C++编程语言中模板函数的概念，但更容易理解。

5. Python语法简介

- **函数：默认参数**

- Python定义函数时，可对部分或全部参数赋默认值，如对以上add函数，可将b赋默认值7。

	带默认参数的函数	1-7
1	**def add(a = 3, b = 7):**　　# 参数b使用默认值	
2	sum = a + b	
3	return sum	
4	# 含有默认参数的函数调用	
5	print(add())　　# 输出结果10，a、b均使用默认值	
6	print(add(5))　　# 输出结果12，a使用输入值，b使用默认值	
7	print(add(4, 5))　　# 输出9，a、b均使用输入值	
8	print(add(a = 1, b = 2))　　# 输出3，a、b均使用输入值，在函数参数中写明参数名	

5. Python语法简介

- **函数：返回多个值**

- Python函数可以一次返回多个值，这也是Python区别其他编程语言（如C++、C#、Java等）的特性之一。

	多个返回值的函数	1-8
1	**def hybrid(a, b):**	# 定义四则运算函数，返回对输入参数加、减、乘、除4个结果
2	return a + b, a - b, a * b, a / b	
3	# 函数调用	
4	x, y, z, w = hybrid(8, 2)	# 结果：x = 10，y = 6，z = 16，w = 4.0
5	print(hybrid(10, 5))	# 输出：(15, 5, 50, 2.0)。输出结果事实上为一个元组

5. Python语法简介

- **类**：属性和方法

- 在Python中定义类需要使用**class**关键字引导，class后跟类名，类名首字符建议使用大写英文字母，类名后跟冒号。下面以People类为例说明类中属性绑定方法。

定义一个People类，但在类中不定义任何东西（空类） 1-9

```
1  class People:
2      pass                    # pass为Python中占位语句，不做任何事情
3  # 类的使用
4  p = People()                # 定义一个People对象p，类名后跟圆括号
5  p.name = 'Jim'              # 给对象绑定name属性，并赋值‘Jim’
6  p.age = 11                  # 给对象绑定age属性，并赋值11
7  print(p.name, p.age)        # 输出：Jim 11
```

- 虽然Python支持这种类属性后期自由绑定的特性，但为了增强代码的**可读性**，建议通过一个特殊的**__init__方法**（**构造函数**，前后两个下划线），在类定义时就将相关属性绑定。

定义一个People类，绑定name、age属性 1-10

```
1  class People:
2      def __init__(self, name, age):                  # __init__函数实现
3          self.name = name                            # 绑定name和age
4          self.age = age
5      def __str__(self):      # __str__函数实现，该函数定义People类对象被print时的行为
6          return '%s: %d' % (self.name, self.age)     # Python字符串格式化
7      def talk(self):                                 # 定义People中的talk行为
8          print('I am %s, %d years old. ' % (self.name, self.age))
9  # 类的使用
10 p = People('Jim', 11)           # 定义一个People对象p，括号内可输入由__init__函数定义的参数
11 print(p)                        # 输出：Jim: 11
12 p.talk()                        # talk函数调用，输出：I am Jim, 11 years old.
```

5. Python语法简介

- **类**：继承

- 在实际生活中，People是一个大类，具体到学校，People又可分为Student（学生）和Teacher（老师）。Student和Teacher都属于People，但各自又有不一样的特征。
- 在面向对象的编程语言中，这种情况一般用类的继承表述。我们可以定义Student类和Teacher类（子类），这两个类都成从People类（父类）继承。

	定义Student和Teacher类，继承自People	1-11
1	**class Student(People):**	
2	**def talk(self):**	# 定义Student中的talk行为
3	print('I am %s, %d years old. I am a student' % (self.name, self.age))	
4	**def learn(self):**	# 对学生定义learn方法
5	pass	
6	**class Teacher(People):**	
7	**def talk(self):**	# 定义Teacher中的talk行为
8	print('I am %s, %d years old. I am a teacher' % (self.name, self.age))	
9	**def teach(self):**	# 对教师定义teach方法
10	pass	
11	# 类的使用	
12	student = Student('Jim', 11)	# 定义一个Student对象
13	student.talk()	# talk函数调用，输出：I am Jim, 11 years old. I am a student.
14	teacher = Teacher ('Tim', 31)	# 定义一个Teacher对象
15	teacher.talk()	# talk函数调用，输出：I am Jim, 31 years old. I am a teacher.

5. Python语法简介

• 模块

- 在Python中，通常将实现某一特定功能的类、函数、变量等进行整合、封装，放在同一个.py文件中，这个文件就称为一个模块。下面创建一个简单的Circle模块，在该模块中实现一些简单的数学运算功能。

Circle.py	Circle模块	1-11

```
1   PI = 3.1415926                          # 定义变量，圆周率π
2   class Cirlce:                           # 定义类， 圆
3     def __init__(self, radius):
4       self. radius = radius               # 绑定radius属性
5     def area(self):                       # 计算圆面积函数
6       return self. radius * self.radius*PI
7   def compare(circle1, circle2):            # 在类外定义函数，比较两圆面积是否相等
8     if circle1.area() != circle2.area():
9       return 'Not equal'
10    else:
11      return 'Equal'
12  # 模块测试
13  if __name__ == '__main__':              # Python main函数写法，模块测试
14    c1 = Cirlce(2)                        # 创建两个圆，半径分别为2和3
15    c2 = Cirlce(3)
16    print(c1.area())                      # 输出：12.5663704
17    print(compare(c1, c2)                 # 输出：Not equal
```

注：可以在模块内写main函数对当前模块进行单元测试。

5. Python语法简介

- **模块：模块使用**
- 在另一个.py文件中使用Circle模块，使用**import Circle**。

Test1.py	模块使用示例1		1-12
1	import Circle	# 使用import关键字导入circle模块	
2	**if __name__ == '__main__':**	# 定义main函数	
3	c1 = Circle.Cirlce(2)		
4	c2 = Circle.Cirlce(3)		
5	print(c1.area())	# 输出：12.5663704	
6	print(Circle.compare(c1, c2)	# 输出：Not equal	
7	print(Circle.PI)	# 输出：3.1415926	

- 使用**from Circle import ***

Test2.py	模块使用示例2		1-13
1	from Circle import *		
2	**if __name__ == '__main__':**	# 定义main函数	
3	c1 =Cirlce(2)		
4	c2 = Cirlce(3)		
5	print(compare(c1, c2)	# 输出：Not equal	
6	print(PI)	# 输出：3.1415926	

注:
from Circle import *语句中的*表示导入Circle模块中所有内容，有时候这是不必要的，也许在Test2.py中只需要使用Circle模块中的PI变量或Circle类，此时只需将上述语句改写成from Circle import PI或from Circle import Cricle即可。

本章小结

- 这一章对Python这门近年来十分火爆的编程语言做了非常简明扼要的介绍。
  - 首先介绍了Python起源、Python解释器及第三方库安装。
  - 在此基础上，介绍了一款常用Python集成开发环境——PyCharm，并在PyCharm中演示了项目创建、代码调试运行等功能。
  - 此外，对Python语法做了简要介绍，包括数据类型、运算符、流程控制、函数、类等。
  - 最后介绍了Python中模块的组织和使用。
- 需要强调的是：**Python只是实现算算法的一种工具**；这一章内容只是希望能尽快带领大家入门，如果需要进一步学习Python知识，请参考相关教材或网站。

课堂练习

1. 练习Pycharm使用；
2. 练习Python语法：数据类型、函数、模块等；
3. 完成练习：打印 1~∞ 内的所有质数；
4. 完成练习：设计一种排序算法，输入随机数列表；
5. 完成练习：People类和Student类，理解传值和传引用；
6. 完成练习：Circle模块，并在main函数中使用。



思考题

1. 简述Python相对于其他编程语言的优缺点。
2. 如何设置Python环境变量？设置Python环境变量的作用是什么？
3. 说明Python第三方库的安装方法及安装目录。
4. 说明Python中元组、列表、字典三种数据结构的表示方法和区别。
5. 简述Python中类和模块的定义方法和区别。
6. 查阅资料，了解Python类中属性后期绑定的实现方法。